CS441 Short notes final (Most important)

Indexing……



Big Data

Definition

Big Data is a term that describes **data sets that are so large, fast-changing, and diverse** that they cannot be managed, processed, or analyzed with traditional data-processing tools (like single-server databases or spreadsheet programs).
These datasets are generated from a **variety of sources**, including:

- Social media platforms (Facebook, Instagram, Twitter)
- Business transactions
- Scientific research instruments
- IoT (Internet of Things) devices
- Video surveillance systems

z**Historical Context**

- **Before 2000** – Data was mostly structured (tables, rows, columns) and stored in RDBMS.
- **2001** – The term “Big Data” gained popularity when Doug Laney introduced the **3Vs model** (Volume, Velocity, Variety).
- **2006** – Apache released Hadoop, inspired by Google’s GFS (Google File System) and MapReduce research papers.

- **2010 onwards** – Explosion of unstructured data from smartphones, social media, IoT.

The 5 Vs of Big Data

Originally there were **3 Vs**, but modern definitions often include **5 Vs** or even more.

1. **Volume** –
    - Data size is massive, ranging from terabytes (10^{12} bytes) to zettabytes (10^{21} bytes).
    - *Example:* Walmart handles over 1 million customer transactions per hour.
2. **Velocity** –
    - The rate at which data is generated and must be processed.
    - *Example:* Financial markets require data updates in microseconds.
3. **Variety** –
    - Data formats:
        - Structured (databases)
        - Semi-structured (XML, JSON)
        - Unstructured (images, videos, text, audio)
    - *Example:* A single weather monitoring system can produce numerical readings (structured) and satellite images (unstructured).

4. **Veracity** –
    - Reliability and accuracy of data.
    - *Example:* Fake reviews on e-commerce sites create “noisy” data.
5. **Value** –
    - The ultimate goal: extract meaningful insights that drive decisions.
    - *Example:* Using purchase data to recommend products to customers.

Diagram in Text Form – Big Data 5 Vs:

```
+-----------+
|  Volume   | --> Massive size
+-----------+
| Velocity  | --> Fast data flow
+-----------+
| Variety   | --> Different formats
+-----------+
| Veracity  | --> Quality & trust
+-----------+
|  Value    | --> Business insights
+-----------+
```

Challenges of Big Data

- **Storage Management** – Traditional single-server databases cannot handle petabytes of data.
- **Processing Speed** – Data must be processed in real-time or near real-time for usefulness.
- **Integration** – Combining different data formats and sources.
- **Security & Privacy** – Sensitive information must be protected.
- **Data Cleaning** – Removing duplicates, errors, and inconsistencies.

Big Data Technologies Overview

Big Data technologies solve three major issues:

1. **Storage** – Distributed file systems like HDFS store massive datasets reliably.
2. **Processing** – Parallel processing frameworks like MapReduce and Spark.
3. **Analysis** – Tools like Hive, Pig, and Machine Learning frameworks.

1The Backbone of Big Data

Introduction to Hadoop

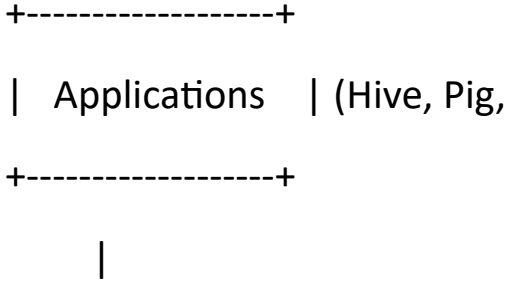
- **Definition:** An open-source framework for distributed storage and processing of big data across clusters of computers.
- **Inspired by:** Google File System (GFS) and MapReduce papers.

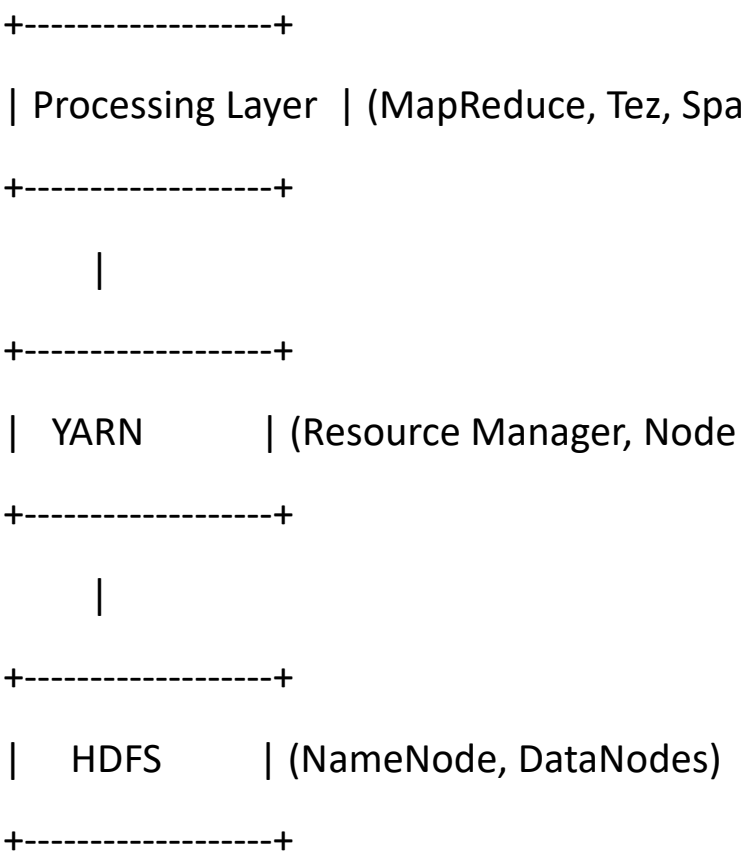
- **Languages Supported:** Java (core), but tools allow SQL-like queries (HiveQL), Python, R.

Hadoop Ecosystem Components

1. **HDFS (Hadoop Distributed File System)** – Storage layer.
2. **YARN (Yet Another Resource Negotiator)** – Resource management and scheduling.
3. **MapReduce** – Processing layer.
4. **Hive** – Data warehousing with SQL-like syntax.
5. **Pig** – Scripting language for analyzing large datasets.
6. **HBase** – NoSQL database.
7. **Sqoop** – Transfer between Hadoop and relational databases.
8. **Flume** – Log data ingestion.
9. **Oozie** – Workflow scheduler.
10. **ZooKeeper** – Coordination service for distributed applications.

Hadoop Architecture:





HDFS (Hadoop Distributed File System)

Purpose: Store very large files reliably and provide high-throughput access to them.

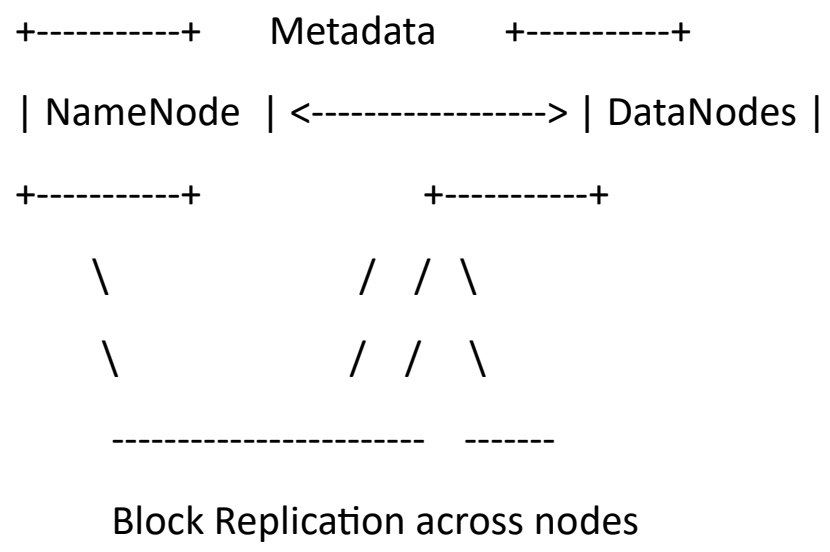
Features

- **Block-based storage** – Files split into fixed-size blocks (default 128MB or 256MB).
- **Replication** – Each block is stored on multiple nodes (default replication factor: 3).
- **Fault tolerance** – If a node fails, data is still accessible from replicas.

Architecture

1. **NameNode** (Master)
    - Stores metadata (file names, block locations).
    - Manages namespace and permissions.
    - Does not store actual data.
2. **DataNodes** (Slaves)
    - Store the actual data blocks.
    - Send periodic heartbeats to NameNode to confirm availability.

Diagram :



MapReduce Workflow

Definition:
A programming model for parallel processing of large datasets, consisting of **Map** and **Reduce** stages.

1.7.1 Steps

1. **Input Splitting** – Dataset split into chunks (input splits).
2. **Mapping** – Map function processes data into (key, value) pairs.
3. **Shuffling & Sorting** – Keys grouped together.
4. **Reducing** – Reduce function processes grouped values.
5. **Output** – Results written back to HDFS.

Example

Task: Count word frequency.

- **Map Phase Output:**

(Hadoop, 1)

(is, 1)

(fast, 1)

(Hadoop, 1)

- **Reduce Phase Output:**

(Hadoop, 2)

(is, 1)

(fast, 1)

Advantages of Hadoop & Big Data Tech

- **Scalability** – Can handle growing data easily by adding nodes.
- **Cost-effective** – Runs on commodity hardware.
- **Fault tolerance** – Data replication ensures no data loss.
- **Flexibility** – Works with all types of data.

Real-World Applications

- **E-commerce** – Personalized recommendations.
- **Banking** – Fraud detection.
- **Healthcare** – Disease prediction models.
- **Telecom** – Network optimization.

Apache Hive

Apache Hive is a **data warehouse infrastructure** built on top of Hadoop. It was developed by Facebook to make querying and managing large datasets easier using a language similar to SQL, called **HiveQL**.

Hive is not a traditional database—it does not support real-time transactions. Instead, it is designed for **batch processing** of large, read-heavy datasets stored in **HDFS**.

Hive works best when:

- Data is very large and stored in distributed form.
- You need to run queries similar to SQL without writing MapReduce code.
- The workload is analytical (data summarization, aggregation, and reporting).

Key Features of Hive

- Uses **HiveQL** (Hive Query Language), which is very similar to SQL, so anyone with SQL knowledge can use it.
- Converts queries into **MapReduce, Tez, or Spark jobs** in the background.
- Supports large-scale **ETL (Extract, Transform, Load)** operations.
- Reads data from HDFS and stores metadata in a **Metastore** (usually backed by MySQL or Derby).
- Suitable for **OLAP** (Online Analytical Processing) but not OLTP (transactional systems).

Components of Hive

1. **Hive Client** – Interface where the user submits queries (CLI, JDBC, ODBC, or Web UI).
2. **Driver** – Compiles, optimizes, and executes HiveQL queries.
3. **Compiler** – Converts HiveQL to MapReduce/Tez/Spark jobs.
4. **Metastore** – Stores metadata about tables, partitions, and schemas.
5. **Execution Engine** – Executes the jobs on Hadoop.
6. **HDFS** – Stores the actual data files.

HiveQL Basics (CRUD Operations)

CRUD stands for **Create, Read, Update, and Delete**. In Hive:

- **Create** means creating databases, tables, or views.
- **Read** means querying data with SELECT statements.
- **Update** and **Delete** are possible, but older Hive versions were read-only. Updates and deletes are available in Hive with **ACID transactions** enabled (since Hive 0.14+).

Creating Databases in Hive

```
CREATE DATABASE sales_db;

CREATE DATABASE IF NOT EXISTS sales_db;

SHOW DATABASES;

USE sales_db;
```

Creating Tables in Hive

Hive supports two types of tables:

- **Managed Tables** – Hive manages both data and metadata. Deleting the table removes the data from HDFS.
- **External Tables** – Metadata is managed by Hive, but data stays in its original location in HDFS. Dropping the table only removes metadata, not the actual data.

Example: Managed Table

```
CREATE TABLE employees (
    emp_id INT,
    name STRING,
    department STRING,
    salary FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Example: External Table

```
CREATE EXTERNAL TABLE employees_ext (
    emp_id INT,
    name STRING,
```

```
    department STRING,
    salary FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/user/hive/external/employees';
```

Loading Data into Hive Tables

You can load data into a Hive table from local storage or HDFS.

From local file system:

```
LOAD DATA LOCAL INPATH '/home/user/empdata.csv'
INTO TABLE employees;
```

From HDFS:

```
LOAD DATA INPATH '/hdfs/path/empdata.csv'
OVERWRITE INTO TABLE employees;
```

- **OVERWRITE** replaces existing data in the table.

Reading Data (SELECT Statements)

```
SELECT * FROM employees;
SELECT name, salary FROM employees WHERE department = 'IT';
SELECT COUNT(*) FROM employees;
```

Updating Data in Hive

(Only works if ACID transactions are enabled)

UPDATE employees SET salary = salary + 1000 WHERE department = 'HR';

Deleting Data in Hive

DELETE FROM employees WHERE emp_id = 101;

TRUNCATE TABLE employees;

Basic Joins in Hive

Hive supports different types of joins:

- **INNER JOIN** – Returns rows where there is a match in both tables.
- **LEFT OUTER JOIN** – Returns all rows from the left table, even if there's no match in the right table.
- **RIGHT OUTER JOIN** – Returns all rows from the right table, even if there's no match in the left table.
- **FULL OUTER JOIN** – Returns rows when there is a match in one of the tables.

Example:

SELECT e.name, d.department_name

```
FROM employees e
JOIN departments d
ON e.department = d.dept_id;
```

Example Workflow in Hive

Imagine you have a CSV file with customer purchase records and you want to:

1. Create a database and table.
2. Load the CSV file into Hive.
3. Run queries to analyze sales.

Steps:

```
CREATE DATABASE sales;
USE sales;

CREATE TABLE purchases (
   order_id INT,
   customer_id INT,
   amount FLOAT,
   order_date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE;

LOAD DATA LOCAL INPATH '/home/user/purchases.csv'
INTO TABLE purchases;

SELECT customer_id, SUM(amount) AS total_spent
FROM purchases
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Advantages of Hive

- **Scalable** – Handles petabytes of data.
- **Familiar syntax** – Similar to SQL, making it easy for data analysts.
- **Extensible** – Supports custom UDFs (User Defined Functions).
- **Integration** – Works with many Hadoop ecosystem tools.

Limitations

- Not suited for real-time queries (latency in MapReduce jobs).
- Updates and deletes are slower than in RDBMS.
- Requires a Hadoop cluster to run.

Introduction

HiveQL (Hive Query Language) is the SQL-like language used to query and manage data stored in Hive tables.

It supports a wide range of **functions** and **query concepts** that make data filtering, grouping, aggregating, and performing advanced analytics possible—without writing low-level MapReduce code.

This topic focuses on:

- Filtering data using WHERE and HAVING
- Grouping results using GROUP BY
- Performing aggregations (SUM, COUNT, AVG, etc.)
- Applying **window functions** for advanced analytics
- Understanding joins and aggregate syntax

Filtering Data in HiveQL

Filtering is the process of selecting only those rows from a dataset that meet specific conditions. Hive uses:

- WHERE clause – Filters rows before grouping/aggregation.
- HAVING clause – Filters rows after aggregation.

Example with WHERE:

SELECT * FROM employees

WHERE department = 'IT' AND salary > 50000;

Here:

- The query will only return employees from the IT department with salaries greater than 50,000.
- This filtering happens **before** aggregation.

Example with HAVING:

SELECT department, AVG(salary) AS avg_salary

FROM employees

GROUP BY department

HAVING avg_salary > 60000;

Here:

- The HAVING clause filters the aggregated results to include only departments where the average salary is greater than 60,000.
- This filtering happens **after** aggregation.

GROUP BY in HiveQL

GROUP BY is used to group rows that have the same values in specified columns into summary rows. Often used with aggregate functions like:

- SUM() – Total of values
- AVG() – Average of values
- MAX() – Highest value
- MIN() – Lowest value

- COUNT() – Number of rows

Example:

SELECT department, COUNT(*) AS emp_count

FROM employees

GROUP BY department;

This will return the total number of employees in each department.

Aggregate Functions in HiveQL

Hive provides several built-in aggregate functions:

- SUM(column) – Total sum of values in a column.
- AVG(column) – Average value of a column.
- COUNT(*) – Total number of rows.
- MAX(column) – Highest value in a column.
- MIN(column) – Lowest value in a column.

Example:

SELECT department, SUM(salary) AS total_salary, AVG(salary) AS avg_salary

FROM employees

GROUP BY department;

This calculates total and average salaries for each department.

Window Functions in HiveQL

Window functions allow you to perform calculations across a set of rows related to the current row without collapsing them into a single output row (unlike normal aggregation).

Syntax:

function_name(column) OVER (PARTITION BY col1 ORDER BY col2)

- **PARTITION BY** – Divides the dataset into partitions for separate calculation.
- **ORDER BY** – Defines the order of rows within each partition.

Common Window Functions:

1. ROW_NUMBER() – Assigns a unique sequential number to rows within a partition.
2. RANK() – Assigns rank with gaps for ties.
3. DENSE_RANK() – Assigns rank without gaps for ties.
4. LEAD(column, offset) – Accesses data from following rows.
5. LAG(column, offset) – Accesses data from previous rows.
6. SUM()/AVG() over a window – Cumulative totals/averages.

Example:

```
SELECT name, department, salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank
FROM employees;
```

Here:

- Each department is a partition.
- Employees are ranked based on salary within their department.

Join & Aggregate Syntax in Hive

Hive supports joins similar to SQL, which can be combined with aggregation for more complex queries.

Example of Join + Aggregation:

```
SELECT d.department_name, COUNT(e.emp_id) AS emp_count
FROM employees e
JOIN departments d
ON e.department = d.dept_id
GROUP BY d.department_name
HAVING emp_count > 5;
```

Explanation:

- Join combines employee and department data.
- GROUP BY groups by department name.
- HAVING filters only departments with more than 5 employees.

Best Practices for Filtering, Grouping, and Aggregation in Hive

- Use WHERE for row filtering before grouping—reduces data early and speeds up queries.

- Use HAVING for filtering aggregated results.
- Always include GROUP BY columns in the SELECT statement when using aggregation (unless you use window functions).
- For large datasets, consider **partitioning** and **bucketing** tables for faster grouping and filtering.
- Be careful with NULL values—aggregates may ignore them.

Real-World Example: Sales Analysis

```
SELECT region, SUM(sales) AS total_sales, AVG(sales) AS avg_sales
FROM transactions
WHERE year = 2024
GROUP BY region
HAVING total_sales > 1000000;
```

Use case:

- Summarizes total and average sales for each region in 2024.
- Only includes regions with more than 1,000,000 total sales.

Hadoop Components

Hadoop is not a single software it is a **framework composed of multiple**

components that work together to store, process, and manage massive datasets in a distributed environment.
At the heart of Hadoop lies **HDFS (Hadoop Distributed File System)**, which is the foundation for data storage. Without HDFS, Hadoop cannot manage big data efficiently because it needs a system that is:

- **Scalable** – Can grow by adding more machines.
- **Fault-tolerant** – Can handle node failures without data loss.
- **Distributed** – Stores data across multiple machines for parallel access.

HDFS (Hadoop Distributed File System)
HDFS is a **Java-based distributed file system** modeled after Google File System (GFS).
Its purpose is to store large files reliably across multiple machines and provide **high throughput** for accessing data.

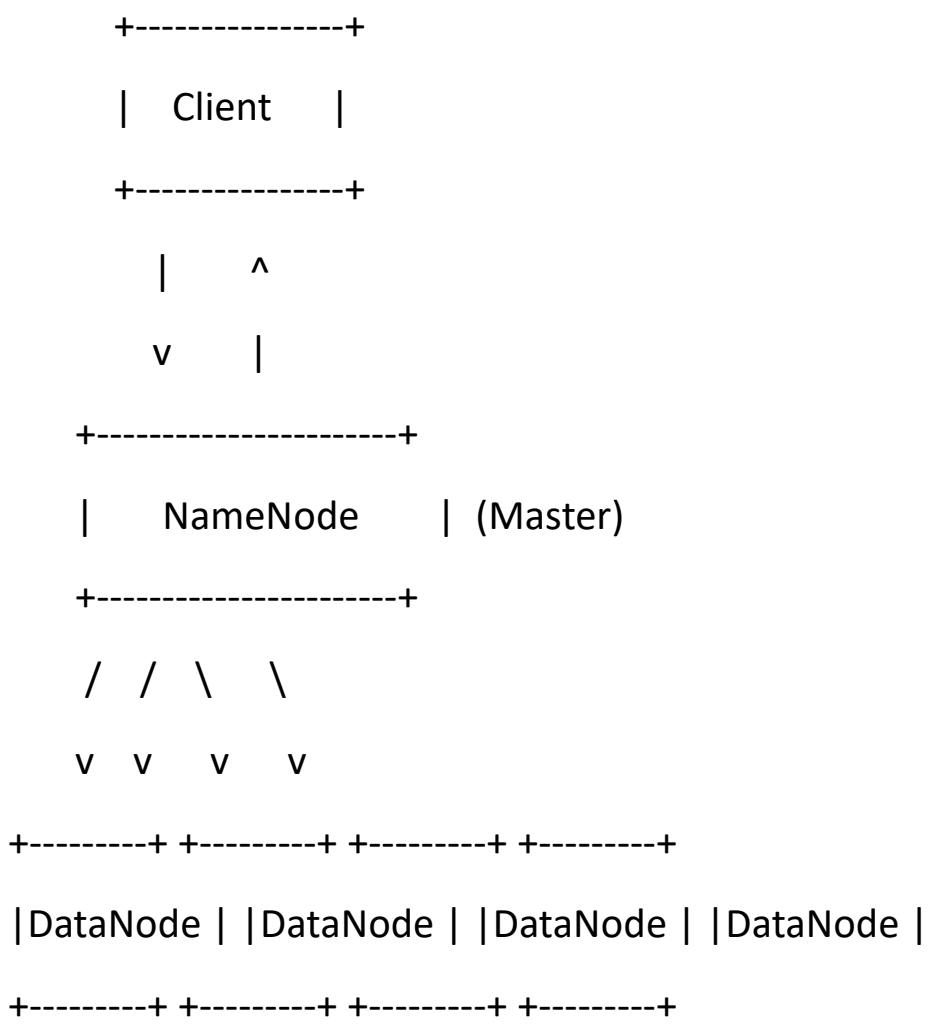
Key characteristics:

- **Stores very large files** (gigabytes to petabytes in size)
- **Streaming data access** rather than random access
- **Optimized for write-once, read-many** workloads (not frequent updates)
- **Runs on commodity hardware** (low-cost servers)

HDFS Architecture Overview
HDFS follows a **Master–Slave architecture**:

- **Master** → NameNode
- **Slaves** → DataNodes

Text Diagram of HDFS Architecture:



NameNode

The NameNode is **the brain of the Hadoop filesystem**.

Its main job is **managing metadata**—the information about where the data is stored, not the data itself.

Responsibilities:

- Keeps the **namespace** of the file system (like a directory tree)
- Records metadata:
    - File names
    - Permissions
    - List of blocks in each file
    - Location of each block (which DataNode stores it)
- Monitors DataNode health via heartbeats
- Handles file creation, deletion, and replication

Important points about NameNode:

- **Single point of control**—if the NameNode is down, HDFS cannot function.
- It **does not store actual data**—only metadata.
- Stored metadata in **RAM** for fast access.
- Uses an **Edit Log** to record every change (used during recovery).

Secondary NameNode vs Backup NameNode:

- Secondary NameNode is **not a backup**—it periodically merges Edit Log with the filesystem image to reduce recovery time.
- High Availability setups use an **Active-Standby NameNode** pair to prevent downtime.

DataNodes – The Workers of HDFS

DataNodes are the **storage units** of HDFS.

Responsibilities:

- Store actual data blocks (default size 128MB each)
- Serve read/write requests from clients
- Send **heartbeats** to NameNode every few seconds
- Send **block reports** (list of blocks stored) to NameNode periodically
- Replicate blocks when instructed by NameNode

DataNode Failure Handling:

- If a DataNode fails (heartbeat not received), NameNode marks it as **dead**.
- NameNode instructs other DataNodes to replicate missing blocks to maintain replication factor.

Fault Tolerance in HDFS

HDFS is designed to handle hardware failures **gracefully** because in a big data environment:

- Machines can fail anytime.
- Network issues can occur.
- Disks can crash.

Fault tolerance is achieved through:

1. **Block replication** – Each data block is stored on multiple DataNodes.
2. **Heartbeat monitoring** – Detects failed DataNodes quickly.
3. **Automatic re-replication** – Lost blocks are recreated from existing replicas.
4. **Rack awareness** – Ensures replicas are stored across different racks to prevent data loss from rack failure.

Block Storage and Replication in HDFS

When a file is stored in HDFS:

1. It is split into **blocks** (default: 128MB).
2. Each block is **replicated** (default replication factor: 3).
3. Replicas are stored across different DataNodes and racks.

Example:

If you store a 300MB file with a block size of 128MB and replication factor of 3:

- File is split into 3 blocks:
    - Block A → Stored on 3 different nodes
    - Block B → Stored on 3 different nodes
    - Block C → Stored on 3 different nodes
- Total physical storage used: 900MB (300MB × 3 replicas)

Rack Awareness in Replication

Rack awareness means that HDFS knows which DataNodes are on the same rack.

By default:

- One replica is stored in the same rack as the client.
- Another replica is stored on a different rack.
- The third replica is stored in the same rack as the second replica.

Benefit: Protects against rack-level failures.

Read and Write Process in HDFS

Write Process:

1. Client requests NameNode to create a file.
2. NameNode checks namespace and returns list of DataNodes for block storage.
3. Client writes block to first DataNode → DataNode pipelines block to second → second sends to third.
4. DataNodes confirm write to NameNode.
5. NameNode updates metadata.

Read Process:

1. Client requests file from NameNode.
2. NameNode returns list of DataNodes with required blocks.
3. Client reads directly from nearest DataNode (network optimization).

4. Blocks are reassembled into the complete file.

Advantages of HDFS

- Highly fault-tolerant (data replication)
- Scalable (add more nodes)
- Cost-effective (commodity hardware)
- High throughput for large files

Limitations of HDFS

- Not suitable for small files (metadata overhead)
- High latency for random reads
- Not optimized for frequent updates

Mapper

In the Hadoop ecosystem, the MapReduce programming model is a core concept used for processing large-scale data in a distributed manner. It consists mainly of three phases:

- **Mapper**
- **Combiner**
- **Reducer**

Mapper

- The Mapper is the first phase in the MapReduce process.
- It takes input data and processes it in parallel, producing intermediate key-value pairs.
- Input to Mapper: Usually, it reads data from HDFS (Hadoop Distributed File System) split into chunks (InputSplits).
- Output from Mapper: Intermediate key-value pairs that are sent to the next phase (Shuffle and Sort).

Role:

- Processes raw input records line by line.
- Extracts meaningful key-value pairs for downstream processing.

Input Format:

- Usually, each Mapper gets a key and a value.
    - Key: Offset of the line in the file (LongWritable in Java)
    - Value: The line content (Text)

Output Format:

- Mapper emits key-value pairs (e.g., <word, 1> in word count example).

Example:

- **Problem:** Word Count
- **Input:** Lines of text
- **Mapper output:** For each word in the line, emit <word, 1>

Combiner

- Combiner is an **optional** phase.
- Sometimes called the "mini-reducer".
- Runs after the Mapper to perform local aggregation of Mapper output before sending data over the network to Reducers.
- Helps reduce network traffic by combining values with the same key.

Role:

- Locally aggregates Mapper output to optimize performance.

When to use:

- When the Reduce operation is associative and commutative (e.g., sum, count).

Example:

- In Word Count, Combiner sums up counts for the same word locally, so instead of sending multiple <word, 1> pairs, it sends <word, count>.

Reducer

- The Reducer takes the intermediate key-value pairs generated by Mappers (and possibly combined by Combiners).
- It processes all values associated with a particular key and produces final aggregated output.

- Input: Key and a list of values.
- Output: Final key-value pairs to be written back to HDFS or another storage.

Role:

- Aggregates, summarizes, or filters data.
- Produces the final result set.

Example:

- **Word Count Reducer:**
    - Input key: word
    - Input values: [1, 1, 1, ...]
    - Output: <word, total_count>

Input/Output Formats

- **InputFormat** defines how input files are split and read.
- Examples:
    - TextInputFormat: Reads lines of text files.
    - KeyValueInputFormat: Reads key-value pairs from text.
- **OutputFormat** defines how the output data is written.
- Examples:
    - TextOutputFormat: Writes output as plain text.
    - SequenceFileOutputFormat: Writes output as binary files.

Word Count

- Input: A text file with multiple lines.
- Mapper Output: <word, 1>
- Combiner (optional): Sums counts for the same word locally.
- Reducer Output: <word, total_count>

Pseudocode for Mapper:

```
for each line in input:
  for each word in line:
    emit(word, 1)
```

Pseudocode for Reducer:

```
for each key (word):
  sum = 0
  for each value in values:
    sum += value
  emit(word, sum)
```

Hive vs Traditional SQL

What is Hive?

Apache Hive is a **data warehouse software built on top of Hadoop**. It provides an SQL-like interface called **HiveQL** to query large datasets stored in Hadoop Distributed File System (HDFS) or other compatible file systems. Hive translates HiveQL queries into MapReduce (or other execution engines like Tez, Spark) jobs that process the data in a distributed way.

What is Traditional SQL?

Traditional SQL (Structured Query Language) is the standard programming language used to manage and query **relational databases** such as MySQL, PostgreSQL, Oracle, SQL Server, etc. These systems store data in structured tables with fixed schemas and support fast transactional processing (OLTP) and analytical querying (OLAP).

Differences between Hive and Traditional SQL

Aspect	Hive	Traditional SQL
Data Storage	Uses Hadoop HDFS or compatible storage.	Uses RDBMS storage engines and disk files.
Schema	Schema on read (applied when reading data).	Schema on write (data validated before storing).
Data Types	Limited and less strict, mainly text-based.	Rich data types and strict enforcement.

Aspect	Hive	Traditional SQL
Query Execution	Translates queries to MapReduce/Spark jobs, batch processing.	Executes queries directly, optimized for fast response.
Latency	High latency; queries take minutes to hours.	Low latency; queries usually complete in milliseconds to seconds.
Transaction Support	Limited or no ACID support (recent versions have some support).	Full ACID compliance (Atomicity, Consistency, Isolation, Durability).
Use Case	Big data batch processing, data warehousing, ETL.	Transactional systems, real-time queries, operational databases.
Joins and Subqueries	Supported but limited and slower due to distributed nature.	Fully supported and optimized.

1. **Data Storage & Schema Management:**
    - In traditional SQL databases, data is stored in tables with a fixed schema. When you insert data, it must conform to the schema, meaning “schema-on-write.”
    - Hive takes a different approach called **schema-on-read**. Data is stored as files (like CSV, JSON, ORC, Parquet) on HDFS without strict schema enforcement. The schema is applied

when you query the data, so it can handle semi-structured or unstructured data more flexibly.

2. **Query Execution and Performance:**
    - Traditional SQL systems process queries directly on the stored data using optimized execution engines that use indexes, query plans, caching, etc., to provide fast results.
    - Hive compiles the query into one or more MapReduce or Spark jobs. This distributed processing is powerful for huge datasets but results in higher latency — queries take longer (minutes to hours), making Hive unsuitable for real-time querying.
3. **Use Cases:**
    - Hive is primarily used for batch processing and large-scale analytics on massive datasets, such as web logs, sensor data, or clickstreams.
    - Traditional SQL databases are used where fast, consistent, real-time access to structured data is required, such as banking systems, e-commerce platforms, or enterprise apps.
4. **Transaction Support:**
    - Traditional SQL databases provide full ACID compliance to ensure data integrity during concurrent access or failures.
    - Hive's ACID support is limited and only available in newer versions with specific storage formats. Hive is generally not used for transactional systems.
5. **Joins and Complex Queries:**

- Traditional SQL engines optimize joins and subqueries with indexes and execution plans.
- Hive supports joins and subqueries but they are slower and more resource-intensive because they translate into distributed computations.

6. **Data Manipulation:**
   - SQL supports fast insert, update, and delete operations.
   - Hive was originally designed for read-only or append-only workloads; update and delete operations are supported only in recent versions and not as efficiently.
7. **Metadata Management:**
   - Hive uses a **Metastore** to store schema and table metadata. This metastore acts like a system catalog.
   - Traditional SQL databases manage metadata internally within the database.
8. **Integration and Ecosystem:**
   - Hive integrates tightly with Hadoop ecosystem tools like HDFS, HBase, Pig, Spark.
   - Traditional SQL databases often integrate with enterprise apps, BI tools, and support standards like ODBC/JDBC.

- **Hive is like a bridge** that lets you write SQL-like queries to analyze massive datasets stored in Hadoop, but it's designed for batch processing where speed isn't critical.

- **Traditional SQL databases** are designed for fast, real-time querying and transaction processing on structured data with strict schema enforcement.

Example to Illustrate

Suppose you want to find the count of users from a dataset.

- In **traditional SQL**, you would run:

SELECT country, COUNT(*) FROM users GROUP BY country;

This query will execute immediately on your relational database if the data is indexed.

- In **Hive**, you run a similar HiveQL query:

SELECT country, COUNT(*) FROM users GROUP BY country;

But under the hood, Hive will translate this query into one or more MapReduce jobs, scanning huge data files in HDFS. It will take longer but can handle terabytes of data spread across many machines.

3. Standard SQL vs HiveQL

What is Standard SQL?

Standard SQL (Structured Query Language) is the ANSI/ISO standardized language used for managing and querying relational databases. It supports a rich set of commands to perform data definition (DDL), data

manipulation (DML), transaction control, and data query operations. Standard SQL is implemented in traditional RDBMS systems like MySQL, PostgreSQL, Oracle, SQL Server, and many others.

What is HiveQL?

HiveQL is the query language used by Apache Hive. It is designed to be similar to SQL for ease of use but adapted to work on big data stored in Hadoop. HiveQL translates queries into distributed jobs (MapReduce, Tez, or Spark) for processing large-scale datasets on clusters.

Key Differences Between Standard SQL and HiveQL

1. Syntax and Features

- Both Standard SQL and HiveQL share many similar syntactical elements such as SELECT, FROM, WHERE, GROUP BY, ORDER BY, JOIN, etc.
- However, HiveQL does not support all Standard SQL features. For example, it has limited support for transactions, updates, and deletes (introduced only in newer versions).
- HiveQL supports additional clauses for big data handling such as PARTITIONED BY and CLUSTERED BY to optimize data storage and querying.
- Standard SQL supports more complex features like window functions, stored procedures, triggers, and recursive queries (with Common Table Expressions), whereas HiveQL's support is limited or missing in these areas.

2. Data Types

- Standard SQL databases support a wide variety of data types including numeric types, character types, date/time types, boolean, binary, etc.
- HiveQL supports a smaller subset of data types optimized for big data formats. It includes primitive types like INT, STRING, FLOAT, and complex types like ARRAY, MAP, and STRUCT to handle semi-structured data.

3. Schema Enforcement

- Standard SQL databases enforce schemas strictly on write (schema-on-write). This means data must conform to the table's schema when inserted.
- Hive uses schema-on-read. Data files stored on HDFS may not strictly conform to schema until read, providing flexibility to query semi-structured or raw data without pre-processing.

4. Query Execution Model

- Standard SQL queries run on a single server or distributed RDBMS with specialized query optimizers, indexes, and caching to ensure fast, low-latency responses.
- HiveQL queries are converted into distributed jobs running on Hadoop clusters, typically with higher latency but scalable to massive datasets.

5. Transaction and ACID Support

- Standard SQL databases provide full ACID compliance ensuring data integrity and consistency during concurrent operations.

- Hive's transaction support (ACID) is limited, mostly available in newer Hive versions with specific storage formats (like ORC). Earlier versions only support append operations efficiently.

6. Join Support

- Standard SQL fully supports complex joins including inner, outer, cross, semi-joins, and anti-joins with optimizations.
- Hive supports joins but performance can degrade significantly for large datasets due to the distributed nature of execution. Some join types like cross joins may be inefficient or unsupported.

Use Cases for Standard SQL

- Applications requiring fast, real-time transactions (banking, e-commerce).
- Systems that need strong data integrity and consistency.
- Operational databases with frequent inserts, updates, and deletes.
- Complex queries with advanced features like window functions or stored procedures.

Limitations

- Scalability challenges with extremely large datasets (multi-terabyte or petabyte scale).
- Expensive hardware required for scaling up.
- Difficulties handling semi-structured or unstructured data natively.

Use Cases for HiveQL

- Batch processing and analysis of very large datasets stored in Hadoop.
- Data warehousing and ETL processes where data volumes are huge and schema flexibility is needed.
- Analyzing semi-structured or log data without extensive preprocessing.
- Use cases where query latency is less critical but scalability is paramount.

Limitations

- Not suitable for real-time or low-latency queries.
- Limited support for updates, deletes, and ACID transactions.
- Complex joins and subqueries can be slow and resource-intensive.
- Requires understanding of Hadoop ecosystem and job optimization.

Hive Tables

In Apache Hive, tables are the primary structures used to organize data. Hive tables can be broadly categorized into two types:

- **Internal Tables**
- **External Tables**

Internal Tables

Definition:
An internal or managed table is a Hive table where Hive **owns the data and metadata**. When you create a managed table, Hive manages both the schema and the data files.

Data Storage:

- The data for internal tables is stored in a subdirectory of the Hive warehouse directory on HDFS (by default, /user/hive/warehouse/tablename).
- Hive controls the lifecycle of these data files.

Data Deletion Behavior:

- When you **drop** an internal table, Hive deletes both the table metadata (schema information) and the **actual data files** stored on HDFS.

- This means that dropping a managed table **removes the data permanently** from the warehouse directory.

Use Cases:

- Suitable when Hive is the sole owner of the data.
- Ideal for temporary or intermediate data where Hive manages data lifecycle fully.
- Preferred when you want Hive to control storage and cleanup.

Example of Creating an Internal Table:

```
CREATE TABLE employees (
  id INT,
  name STRING,
  salary FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

In this case:

- Data is stored inside Hive's managed warehouse.
- Loading data into the table copies the data files into the warehouse.

External Tables

Definition:
An external table is a Hive table where Hive **only manages the metadata**, but the **data itself is stored externally** Hive does not own or control the data files.

Data Storage:

- The data files reside **outside** the Hive warehouse directory, at a location specified by the user during table creation.
- This location can be anywhere in HDFS or compatible storage, such as Amazon S3.

Data Deletion Behavior:

- When you **drop** an external table, Hive deletes only the metadata associated with the table.
- The actual data files on HDFS **remain intact**.
- This ensures that external data is safe and accessible outside of Hive.

Use Cases:

- Suitable when data is shared with other applications or tools outside Hive.
- Preferred for datasets that are managed externally and should persist even if Hive tables are dropped.
- Useful for large datasets where data copying is expensive or undesirable.

Example of Creating an External Table:

```
CREATE EXTERNAL TABLE logs (
```

```
  event_id STRING,
  event_timestamp TIMESTAMP,
  user_id STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
LOCATION '/user/data/logs/';
```

In this case:

- Data files exist at /user/data/logs/ on HDFS.
- Dropping the table removes only the Hive metadata, not the data files.

Key Differences

Ownership:

- Internal: Hive owns both data and metadata.
- External: Hive owns only metadata; data is external.

2. **Data Location:**
   - Internal: Data stored inside Hive's managed warehouse directory.
   - External: Data stored at user-specified external location.
3. **Data Lifecycle on Drop:**
   - Internal: Dropping deletes both metadata and data files.

    - External: Dropping deletes metadata only; data files remain.
4. **Data Loading:**
    - Internal: Loading data copies files into Hive warehouse directory.
    - External: No data copy; Hive references data at the external location.
5. **Use Cases:**
    - Internal: When Hive controls entire lifecycle and ownership.
    - External: When data is shared, persistent outside Hive, or managed by other tools.

Additional Important Points

- **Switching Table Types:**
  You can convert an internal table to external or vice versa by altering the table property EXTERNAL in Hive. However, be cautious because changing the ownership affects data deletion behavior.
- **Data Safety:**
  External tables are safer when multiple systems access the same data because dropping the table does not delete data.
- **Partitioning:**
  Both internal and external tables support partitioning to improve query performance by dividing data into smaller parts based on column values.

- **File Formats:**
  Both table types support various file formats (TextFile, ORC, Parquet, Avro), but external tables commonly point to optimized storage formats shared across tools.

What Happens Internally When You Drop Tables?

- **Drop Internal Table:**
  Hive deletes the table metadata from the metastore and **deletes the data directory** from HDFS (e.g., /user/hive/warehouse/tablename).
- **Drop External Table:**
  Hive deletes only the metadata in the metastore. The actual data files remain untouched at their original location.

Why Does This Matter?

- If you accidentally drop an internal table, you lose both metadata and data permanently unless you have backups.
- Dropping external tables is safer because data persists beyond Hive's control.
- Choosing between internal and external depends on data governance, ownership, sharing needs, and lifecycle management.

Big data ecosystems, like Hadoop with Hive, enable the processing and querying of massive datasets that traditional databases struggle to handle. At the core of this ecosystem is the **MapReduce programming model**, which breaks down large data processing jobs into three phases:

Mapper, **Combiner**, and **Reducer**. The Mapper extracts key-value pairs from raw data, the Combiner optimizes data transfer by local aggregation, and the Reducer aggregates final results.

To make big data querying accessible, **Hive** provides an SQL-like language called **HiveQL**, allowing users familiar with traditional SQL to interact with data stored on distributed file systems. Unlike traditional SQL databases that use **schema-on-write** and provide low-latency transaction support, Hive employs **schema-on-read** and translates queries into distributed jobs suited for batch processing of enormous datasets.

Within Hive, data is organized into two types of tables:

- **Internal tables**, where Hive controls both data and metadata, and deleting the table deletes the data.
- **External tables**, where Hive manages only the metadata, leaving the data files intact even if the table is dropped, facilitating shared or persistent data management.

..... *Sahilkhan.....*